

# Dynamic Exponent-Window Accumulation with Activation Top-2 Selection for Efficient Low-Bit BFP LLM Inference

Ming-Hong Lin

National Taiwan University of Science and Technology  
Department of Electrical Engineering  
Taipei, Taiwan

## Abstract

Block Floating Point (BFP) enables low-cost fixed-point multiply-accumulate within each block, but inter-block accumulation still requires an expensive floating-point accumulator (FP-Acc) when shared exponents differ. In synthesized low-bit BFP processing elements, the FP-Acc can consume up to 75% of power and 55% of area. Activation outliers further widen the exponent dynamic range and amplify FP-Acc activity. We propose Dynamic Exponent-Window Accumulation (DEWA), which confines most partial sums to a low-cost accumulator with a narrow dynamic range before forwarding exceptions to the FP-Acc. To mitigate outlier-induced window shifting, we combine DEWA with activation-only bi-exponent encoding and a Top-2 outlier cap per block, while keeping weights in single-exponent BFP. On LLaMA2-7B with WikiText-2, the proposed W-BFP4 / T2A-BiE4 configuration preserves accuracy within 0.02 perplexity of a strong Top-2 baseline while substantially reducing FP-Acc utilization in functional simulation.

## CCS Concepts

• **Computer systems organization** → **Neural networks**; • **Hardware** → *Arithmetic and datapath circuits*.

## Keywords

Block floating point, large language models, quantization, hardware accelerator, outlier handling

## ACM Reference Format:

Ming-Hong Lin. 2026. Dynamic Exponent-Window Accumulation with Activation Top-2 Selection for Efficient Low-Bit BFP LLM Inference. In *63rd ACM/IEEE Design Automation Conference (DAC '26)*, July 26–29, 2026, Long Beach, CA, USA. ACM, New York, NY, USA, 4 pages. <https://doi.org/XXXXXXX.XXXXXXX>

## 1 Introduction

Low-precision quantization is essential for deploying large language models (LLMs) under tight memory and energy budgets. Block Floating Point (BFP) offers a favorable accuracy–cost trade-off by assigning a shared exponent to each fixed-size block while storing only low-bit mantissas [6, 10]. Recent BFP accelerators

perform intra-block multiply–accumulate (MAC) with fixed-point arithmetic, but inter-block partial sums with mismatched exponents must still be merged in a floating-point accumulator (FP-Acc) [3–5].

Two coupled bottlenecks limit the efficiency of BFP-based LLM inference. First, the FP-Acc dominates processing-element (PE) cost: in synthesized low-bit configurations it can account for up to 75% of power and 55% of area [3–5]. Second, activation outliers compress the effective mantissa of co-located normal values and widen the exponent spread of partial sums, increasing FP-Acc activity [2, 7, 11, 12].

This paper develops a hardware-friendly co-design that addresses both bottlenecks jointly. Our contributions are:

- **Dynamic Exponent-Window Accumulation (DEWA):** a low-cost accumulator that retains partial sums within a sliding exponent window and forwards only out-of-window updates to the FP-Acc.
- **Activation-only bi-exponent encoding with Top-2 selection:** weights remain single-exponent BFP while activations use a bounded bi-exponent format that retains at most two outliers per block, reducing storage and MAC control overhead relative to full BiE [11, 12].
- **Evaluation on LLaMA2-7B:** we quantify perplexity, outlier occupancy, and FP-Acc request reduction under a consistent WikiText-2 protocol, showing negligible accuracy loss for the proposed W-BFP4 / T2A-BiE4 design point.

The remainder of this paper reviews BFP deployment challenges (Section 2), presents DEWA and Top-2 activation routing (Section 3), describes the BFP-PE architecture (Section 4), reports experimental results (Section 5), and concludes (Section 6).

## 2 Background

### 2.1 Block Floating Point

Low-precision integer quantization reduces storage and arithmetic cost, but a single tensor-level scaling factor can be dominated by rare large-magnitude values. This issue is particularly severe in LLM activations, where a small number of outliers may compress most nonoutlier values into a limited set of quantization levels and degrade low-bit accuracy [1, 8, 9].

Block Floating Point (BFP) provides a more fine-grained accuracy–cost tradeoff by partitioning tensors into fixed-size blocks and assigning an independent shared exponent to each block. Each element stores only a sign and a low-bit mantissa (Fig. 1(a)), while the exponent overhead is amortized across the block. Intra-block computation requires only fixed-point multiply–accumulate operations, enabling BFP to achieve higher accuracy at a cost comparable to integer quantization [6, 10].

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](mailto:permissions@acm.org).  
DAC '26, Long Beach, CA, USA

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.  
ACM ISBN 978-x-xxxx-xxxx-x/YY/MM  
<https://doi.org/XXXXXXX.XXXXXXX>

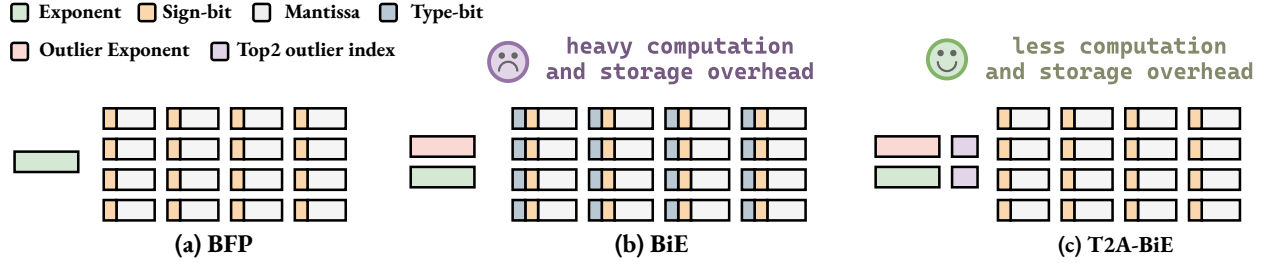


Figure 1: Block-level data formats. (a) Conventional BFP with one shared exponent per block. (b) Bi-exponent BFP (BiE), which adds an outlier exponent and spends a type bit on every element. (c) The proposed Top-2 Activation-only Bi-Exponent BFP (T2A-BiE), which keeps both exponents but replaces the per-element type bits with two outlier indices, so the storage and MAC overhead is lower than that of BiE; weights remain single-exponent BFP as in (a).

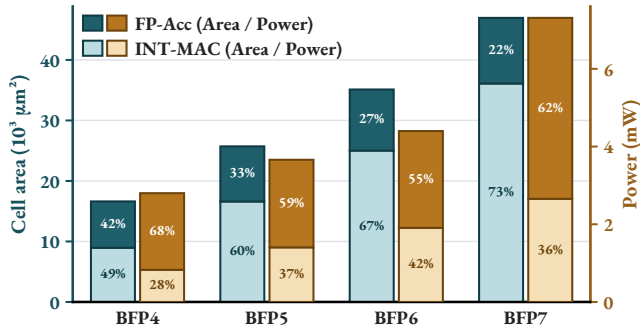


Figure 2: Component-level power and area breakdown of a synthesized baseline BFP-PE, showing FP-Acc dominance.

## 2.2 Deployment Challenges

Applying BFP to LLM inference acceleration introduces two major bottlenecks.

(1) **Inter-block floating-point accumulation.** A state-of-the-art BFP processing element (BFP-PE) can perform MAC operations within each block using low-bit fixed-point arithmetic. However, shared exponents differ across blocks, so partial sums cannot be directly accumulated in the integer domain. An FP accumulator (FP-Acc) is therefore required to perform inter-block exponent alignment and floating-point accumulation. The FP-Acc typically includes alignment, normalization, addition, and fixed-point-to-floating-point conversion logic. In synthesized low-bit BFP-PE configurations, the FP-Acc accounts for up to 75% of PE power and 55% of its area (Fig. 2), making it a primary obstacle to improving the energy and area efficiency of BFP accelerators [3–5].

(2) **Activation outliers.** BFP confines the influence of an outlier to its local block but does not eliminate the outlier itself. When a block contains a large-magnitude outlier, the shared exponent is determined by that value, and nonoutlier values in the same block lose a substantial number of effective mantissa bits [2, 7, 11, 12]. This loss becomes more severe as the mantissa width decreases (Fig. 3).

These two bottlenecks are coupled. Outliers degrade the precision of nonoutlier values during quantization and can also widen the

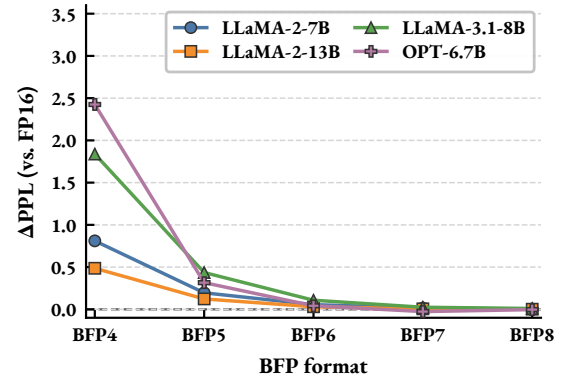


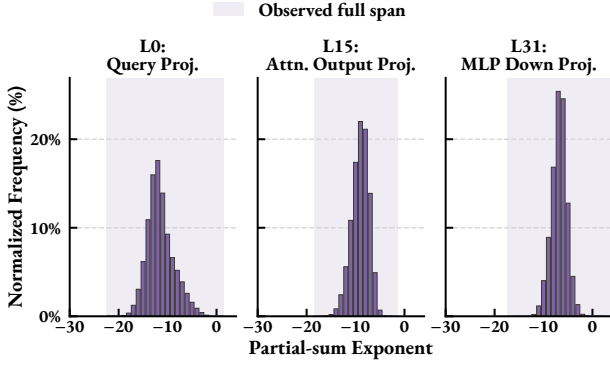
Figure 3: WikiText-2  $\Delta PPL$  of vanilla BFP versus FP16 as the mantissa width decreases (group size 16).

exponent distribution of resulting partial sums, increasing the likelihood that inter-block updates require the FP-Acc. Consequently, effective outlier handling has the potential to improve numerical fidelity while simultaneously reducing FP-Acc activity. This observation motivates the approach developed in this work.

## 2.3 Bi-Exponent BFP for Outlier Separation

A common remedy for the precision loss described above is to give outliers their own exponent instead of forcing an entire block to share one. BiE [12] and Oiso [11] store two shared exponents per block (BiE format), one for nonoutlier values and one for outliers, and prepend a type bit to each element that selects which exponent applies (Fig. 1(b)). Nonoutlier values therefore keep their effective mantissa bits even when the block contains a large-magnitude value.

The accuracy gain, however, comes with non-trivial overhead. Because outliers are inherently rare, a per-element type bit spends storage on every element to describe a small minority, and the two exponents must be applied to both operands of a MAC, yielding four exponent-pair cases that the datapath has to support. This overhead is what makes a direct combination of bi-exponent BFP



**Figure 4: BFP4 partial-sum exponent distributions on three LLaMA2-7B layers (group size 16). The shaded band is the observed full span.**

with a low-cost accumulator unattractive, and it is the starting point for the format we propose in Section 3 (Fig. 1(c)).

### 3 Dynamic Exponent-Window Accumulation

#### 3.1 Partial-Sum Concentration and DEWA

We observed the distribution of partial sums in LLaMA2-7B (Fig. 4). The distribution remains highly concentrated, indicating that a low-cost accumulator with a narrow dynamic range can be introduced prior to forwarding data to the FP-Acc.

Specifically, we define an accumulation window

$$[E_{\max} - T_d, E_{\max} + T_r] \quad (1)$$

where  $E_{\max}$  denotes the exponent of the current maximum partial sum, and  $T_d$  and  $T_r$  represent the drop and replace thresholds, respectively. When a new partial sum arrives:

- if its exponent falls below the lower bound, it is discarded;
- if it exceeds the upper bound, it directly replaces the value currently held in the accumulator;
- otherwise, it is accumulated using the low-cost accumulator.

This approach substantially reduces FP-Acc utilization and thereby lowers power consumption [4, 5]. We refer to this method as **Dynamic Exponent-Window Accumulation (DEWA)**.

Although the majority of partial sums are concentrated, the overall distribution still exhibits a large dynamic range. We attribute this to the influence of outliers, which cause the dynamic exponent window to shift significantly and thereby induce precision loss. We therefore argue that outlier separation is essential as a complement to DEWA. By separating activation outliers, a substantially higher proportion of partial sums fall within the accumulation window.

#### 3.2 Activation Top-2 Selection

Outlier separation is therefore required, but the bi-exponent format of Section 2.3 cannot be dropped into DEWA as is: its per-element type bits and four MAC cases add cost that the rare occurrence of outliers does not justify. We instead retain the two exponents and remove the overhead through two changes:

**Table 1: Outlier occupancy per G16 activation block (W-BFP4/A-BiE4, LLaMA2-7B).**

| Outliers per block | Block count    | Block percentage |
|--------------------|----------------|------------------|
| 0                  | 21,437,737,318 | <b>88.60%</b>    |
| 1                  | 2,373,974,147  | <b>9.81%</b>     |
| 2                  | 241,189,169    | <b>1.00%</b>     |
| $\geq 3$           | 141,941,990    | 0.59%            |

- (1) Since the majority of outliers concentrate on specific activation channels, applying bi-exponent encoding to weights yields benefits that do not justify the associated overhead. We therefore adopt single-exponent BFP for weights while retaining bi-exponent encoding for activations.
- (2) We further examined the number of outliers occurring within each block (Table 1). 99.4% of G16 activation blocks contain at most two outliers. We propose **Activation Top-2 Selection**, wherein at most the two largest outliers are selected per block, further reducing hardware overhead.

We call the resulting activation encoding **Top-2 activation-only bi-exponent BFP (T2A-BiE)**, shown in Fig. 1(c): the two indices identify which lanes of the G16 block use the outlier exponent, so no element carries a type bit. Paired with 4-bit mantissas, the full configuration is denoted W-BFP4 / T2A-BiE4.

Activation-only BiE and Top-2 capping have negligible impact on LLaMA2-7B perplexity relative to full BiE.

### 4 Hardware Architecture

Fig. 5 shows the BFP-PE that implements W-BFP4 / T2A-BiE4 at group size 16. Weights remain single-exponent BFP. Each activation block arrives with two outlier indices instead of per-element type bits. An outlier-index encoder and dispatcher then split the 16 lanes into a normal path and at most two outlier lanes.

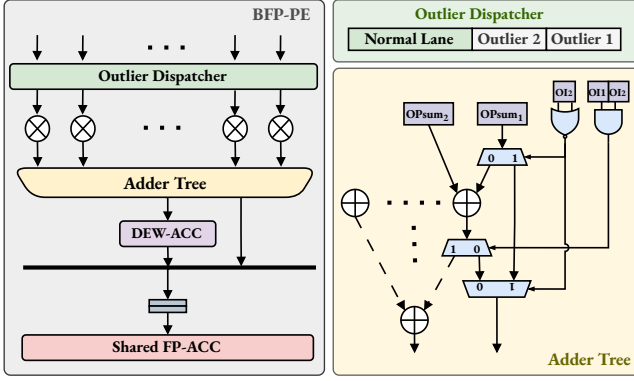
Normal lanes feed an integer adder tree, and the tree output updates the DEWA accumulator of Section 3 under  $(T_{\text{skip}}, T_{\text{replace}})$ . Each occupied outlier lane has its own adder-tree output and is sent directly to a shared FP-Acc; the exception path does not compare exponents. If a block has fewer than two outliers, a reserved index marks the unused slot and that lane contributes a zero partial.

The FP-Acc is therefore shared by the encoded outlier lanes and by DEWA results that leave the integer window, rather than by every inter-block partial sum.

### 5 Evaluation

#### 5.1 Experimental Setup

We evaluate LLaMA2-7B on the WikiText-2 test split using a non-overlapping 2048-token protocol (166 blocks, 339,802 loss tokens). Fake-quantized weights and activations are dequantized before F.linear; attention internals, LayerNorm, and lm\_head remain in FP16 unless noted. Group size is G16 for the primary design point. BiE-family outlier detection uses  $\text{threshold}(X) = \text{mean}(X) + 3 \text{std}(X)$  over signed tensor values.



**Figure 5: Proposed BFP-PE for W-BFP4 / T2A-BiE4 (group size 16). Outlier indices dispatch each block onto a normal DEWA path and up to two outlier lanes that share one FP-Acc.**

**Table 2: WikiText-2 perplexity on LLaMA2-7B (G16, BFP4/BiE4 family).**

| Configuration     | PPL   |
|-------------------|-------|
| FP16 baseline     | 5.472 |
| Vanilla W/A-BFP4  | 6.283 |
| W/A-BiE4          | 5.974 |
| W-BFP4 / A-BiE4   | 6.013 |
| W-BFP4 / T2A-BiE4 | 6.016 |

## 5.2 Accuracy Results

Table 2 compares representative quantization formats against an FP16 reference. Activation-only BiE and Top-2 capping recover most of the accuracy gap of vanilla BFP4 while keeping weights in lower-cost single-exponent BFP.

## 5.3 DEWA Threshold Sweep

Perplexity remains near the Top-2 baseline (6.016) across a wide range of asymmetric DEWA thresholds under W-BFP4 / T2A-BiE4, indicating that aggressive FP-Acc bypassing is compatible with negligible accuracy loss.

## 5.4 Hardware-Oriented Metrics

Functional simulation reports DEWA out-of-window (OOW) decisions and optimistic FP-Acc request counts. RTL synthesis of the baseline BFP-PE confirms FP-Acc dominance (Fig. 2); synthesized area and power for the full Top-2 DEWA design remain future work.

## 6 Conclusion

BFP accelerators face coupled challenges from expensive inter-block FP accumulation and activation outliers that shift shared exponents and widen partial-sum dynamic range. We presented Dynamic Exponent-Window Accumulation (DEWA) to confine most partial sums to a low-cost accumulator, complemented by activation-only bi-exponent encoding with a Top-2 outlier cap while keeping

weights in single-exponent BFP. On LLaMA2-7B, the proposed W-BFP4 / T2A-BiE4 configuration preserves WikiText-2 perplexity within 0.02 of a strong Top-2 baseline while enabling substantial FP-Acc bypass in functional evaluation.

Future work includes RTL implementation and synthesis of the full Top-2 DEWA datapath, cycle-accurate FP-Acc utilization measurement, and extension to additional models and group sizes.

## References

- [1] C. Guo et al. 2023. OliVe: Accelerating Large Language Models via Hardware-Friendly Outlier-Victim Pair Quantization. In *Proc. 50th Annu. Int. Symp. Computer Architecture (ISCA)*. 1–15. doi:10.1145/3579371.3589038
- [2] X. Han et al. 2025. BBAL: A Bidirectional Block Floating Point-Based Quantization Accelerator for Large Language Models. In *Proc. 62nd ACM/IEEE Design Automation Conf. (DAC)*. 1–7.
- [3] X. Ju et al. 2025. SmartBlock: Adaptive Block Floating Point Quantization for Efficient DNN Acceleration. In *Proc. 54th Int. Conf. Parallel Processing (ICPP)*. 428–438. doi:10.1145/3754598.3754660
- [4] X. Ju et al. 2025. WinAcc: Window-Based Acceleration of Neural Networks Using Block Floating Point. In *Proc. Design, Automation & Test in Europe Conf. (DATE)*. 1–7. doi:10.23919/DATE64628.2025.10992780
- [5] Y.-C. Lo and R.-S. Liu. 2023. Bucket Getter: A Bucket-Based Processing Engine for Low-Bit Block Floating Point (BFP) DNNs. In *Proc. 56th IEEE/ACM Int. Symp. Microarchitecture (MICRO)*. 1002–1015. doi:10.1145/3613424.3614249
- [6] B. D. Rouhani et al. 2020. Pushing the Limits of Narrow Precision Inferencing at Cloud Scale with Microsoft Floating Point. In *Advances in Neural Information Processing Systems 33 (NeurIPS)*.
- [7] X. Wang, J. Li, Y. Sun, and W. He. 2026. Harmonia: Algorithm-Hardware Co-Design for Memory- and Compute-Efficient BFP-Based LLM Inference. *arXiv preprint arXiv:2602.04595* (2026). doi:10.48550/arXiv.2602.04595
- [8] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han. 2023. SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models. In *Proc. 40th Int. Conf. Machine Learning (ICML) (PMLR, Vol. 202)*. 38087–38099.
- [9] A. H. Zadeh, I. Edo, O. M. Awad, and A. Moshovos. 2020. GOBO: Quantizing Attention-Based NLP Models for Low Latency and Energy Efficient Inference. In *Proc. 53rd IEEE/ACM Int. Symp. Microarchitecture (MICRO)*. 811–824. doi:10.1109/MICRO50266.2020.00071
- [10] S. Q. Zhang, B. McDanel, and H. T. Kung. 2022. FAST: DNN Training Under Variable Precision Block Floating Point with Stochastic Rounding. In *Proc. IEEE Int. Symp. High-Performance Computer Architecture (HPCA)*. 846–860. doi:10.1109/HPCA53966.2022.00067
- [11] L. Zou et al. 2026. Oiso: Outlier-Isolated Data Format for Low-Bit Large Language Model Quantization. *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.* 45, 2 (Feb. 2026), 929–942. doi:10.1109/TCAD.2025.3585023
- [12] L. Zou, W. Zhao, S. Yin, C. Bai, Q. Sun, and B. Yu. 2024. BiE: Bi-Exponent Block Floating-Point for Large Language Models Quantization. In *Proc. 41st Int. Conf. Machine Learning (ICML) (PMLR, Vol. 235)*. 62978–62992.